

TRIPTI

System architecture

A report typed in panic becomes a truck on the road. This document describes how, and — more importantly — which decisions the software is not allowed to make on its own.

Layer	What is there
Backend	59 files · Node 24 · Express 5 · TypeScript ESM
Frontend	59 files · React 19 · Vite 8 · Tailwind v4
Database	Supabase Postgres · 15 tables · 13 migrations
Tests	182 passing across 12 files, concentrated in the engine
Total	~17,300 lines across backend, frontend and SQL

The rule the whole design serves

The model reads. The engine decides. A human approves. Extraction turns free text into fields; allocation is computed by pure, unit-tested functions that can explain any match term by term; and nothing dispatches without a coordinator saying yes.

01 The pipeline

Seven stages. Each one either adds a fact or asks a person to confirm one.

INTAKE Free text in	EXTRACT Fields + confidence	TRIAGE Human corrects	MATCH Scored pairings	RESERVE Stock held	DELIVER Volunteer runs it	VERIFY Counted
-------------------------------	---------------------------------------	---------------------------------	---------------------------------	------------------------------	-------------------------------------	--------------------------

Only the last stage produces impact figures. A mission marked delivered but never verified is not evidence anything reached anyone, so it is counted as nothing.

02 Trust model — two clients, two levels

The single most consequential decision in the system. There are two Supabase clients and they are not interchangeable.

Client	Holds	Protected by
Browser	Anon key only	Row-level security on every table it touches
Backend	Service role key	Nothing — it bypasses RLS entirely

Why this matters

The service-role client bypasses every policy in the database. Any route using it must therefore authorise itself, in code, before touching a row. Adding a table means adding its policies in the same change — and roles are read from the `profiles` table, never from a JWT claim or a request header a caller could set.

03 The decision engine

Seven pure modules under `backend/src/engine/`. No I/O, no network, no database — inputs in, decision out. This is what makes allocation explainable months later, and it is where the test suite is concentrated.

Module	Decides
priority.ts	How urgent a need is — 0-100, four bands, per-term breakdown
match.ts	Which resource serves which need, and by how much
assignment.ts	Which volunteer and vehicle, on distance and workload
inventory.ts	What is actually free once reservations are counted
missionLifecycle.ts	Which transitions are legal, and who may make them
reallocation.ts	What may be taken from one need and given to another
impact.ts	What was achieved, counted from verified deliveries only

Match weights, from the product brief: need priority 30%, proximity 25%, quantity fit 20%, time fit 15%, transport fit 10%. They sum to one and are configurable in one place.

04 Concurrency — why allocation lives in SQL

Two coordinators planning at the same moment must not promise the same crate twice. Reading stock and then writing a reservation leaves a gap between the two, so the check and the write happen in a single statement inside a database function.

Function	Guarantees
reserve_resource	Holds units only if they are still free at that instant
release_resource	Returns held units when a run fails or is cancelled
consume_resource	Removes units on delivery — stock actually leaves the pool
assign_mission	One volunteer wins a race for the same run
reallocate_match	Moves a reservation between needs without ever unholding it

Proven, not assumed

Five simultaneous requests for 3,000 units against 11,400 free were fired at the live database. Exactly three succeeded. The other two were refused, not silently over-committed.

05 The AI layer, and its limits

Everything the model does sits behind a provider interface with a deterministic fallback. With no API key configured, intake still works: a keyword scan fills what it can, marks confidence zero, and flags every incident for manual triage.

The model may	The model may never
Extract fields from free text	Rank, allocate or assign anything
Classify category and severity	Invent a coordinate
Summarise and translate	Advance a status past open
Flag what it could not read	Supply facts about the world

Coordinates are never inferred

Extraction copies a place name into `location_text` and leaves latitude and longitude null. Geocoding is a separate, explicit step that returns candidates for a coordinator to choose from. A need without coordinates is excluded from planning and returned in a list for manual handling — never quietly dropped, and never guessed. A wrong coordinate sends a truck to the wrong place.

06 Data model

Fifteen tables. The shape worth knowing is the chain from a report to a delivery.

Table	Holds
incidents	The report, plus AI-extracted fields with confidence beside them
needs	What an incident requires — kind, quantity, unit, status
resources	Donated supply, with expiry, perishability and reserved quantity
matches	A scored pairing of need and resource; the reservation itself
missions	A dispatched match, with route, assignee and lifecycle status
status_history	Append-only. Never updated, never deleted
reallocations	What was proposed, what was protected, who approved it

Append-only history

`status_history` is written and never changed. A review weeks later has to be able to reconstruct what was known at the moment a decision was made — not what turned out to be true afterwards.

07 Realtime

Eleven tables publish changes. Every board subscribes and refetches; when the socket drops it falls back to polling every fifteen seconds and says so on screen.

Events are a signal, not data

A realtime event carries the row as the database saw it — not as this user is allowed to see it. Rendering the payload directly would show a volunteer missions that are not theirs. Every event is treated as "something changed, ask again", so authorisation stays in the path written for it.

A stale board that looks live is worse than a slow one

The websocket authenticates as anonymous unless handed the user's token, and row-level security then filters every event away — while the channel still reports a healthy subscription. The screen looked live and received nothing. Each board now states its own connection state rather than implying one.

08 External integrations

Adapter	Service	Behaviour when it fails
geocoding.ts	OpenStreetMap Nominatim	Returns candidates only; never writes a coordinate
routing.ts	OSRM	Falls back to a straight line, drawn dashed to say the roads are unknown
disasterFeed.ts	GDACS (UN / EC)	Serves the last good copy, clearly marked stale

Alerts from the external feed are drawn dashed and hollow, beneath everything else, and excluded from the map's auto-fit. A coordinator must never confuse what the world is reporting with what this team has taken on: one is context, the other is a commitment with people attached.

09 Deployment

Piece	Platform	Why
Database + Auth	Supabase	Postgres, RLS and realtime in one place
API	Render	A long-running server, which is what the code assumes
Web app	Vercel, root frontend	Static bundle on a CDN

The API is deliberately not serverless

Rate limits are held in process. On one long-lived instance the documented ceilings are real; spread across serverless instances each gets its own, and the intake limit that exists to stop an account looping on a paid model endpoint stops meaning what it says.

10 Decisions on record

Decision	Reasoning
Engine is pure, not a service	A coordinator may have to defend an allocation months later. Pure functions can be replayed exactly
Impact recomputed, never accumulated	A counter drifts. Correct a mission and the total still carries the old figure, with nothing to show it should not
People deduped per incident	Three deliveries to one flood of 300 helped 300, not 900
Mixed units report no total	Six bottles and four litres of water is not "ten" of anything
Reallocation fails closed	An unidentifiable donor, a named volunteer, a life-critical need — each refuses the move rather than risking it

Generated from the repository at commit 70df09b. Figures in this document were read out of the code and the live database, not restated from an earlier draft.